Sign-Language HUD Glasses

Hardware bring-up guide: test every part before building.

Branch `testout` · 2026-10-24, updated 2026-10-29 · Charan · Pictures are drawn diagrams, not photos.

Where you are

Steps 1–6 ✅ done. **Next: Step 7** (chargers, USB only), then **Step 9** (solder battery wires) and **Step 10** (run on battery). Charging = bench DC supply only (Step 8).

How to use this guide

1. Do the steps in order: 1 → 15.
2. Do one small action at a time.
3. Check the green **Done when** box before moving on.
4. If something fails, read the orange **If it fails** box.
5. Write every result in the checklist on the last page.

> ⚠ **Safety: read first**
>
> - **Never plug USB into a TP4056 that has a LiPo on it.** Stock board charges at 1 A, too much for 600 mAh. Charge only with the bench supply (Step 8).
> - **Never hold battery wires on the pads by hand.** Solder them (Step 9). A slip shorts the battery directly.
> - Keep the LiPo's red and black wire ends apart. Never let them touch.
> - Meter red lead for volts goes in **Input HI (VΩ)**, never the **10A** jack.
> - Don't let the two meter probes touch each other while a board is powered.
> - Soldering iron (Steps 9, 13) is ~330 °C. Always put it back in its stand.

Parts

In hand ✅

- ESP32-C3 SuperMini ×2
- 0.96" OLED SSD1306 ×2 (4-pin I2C)
- TP4056 Type-C charger ×2
- Slide switch ×3, veroboard 6×4 ×2
- Pin strips, MB102 breadboard, jumpers
- Type-C cable, soldering kit + meter
- LiPo 3.7 V 600 mAh ×2 (KP 602030P)
- 25 mm lens (f 45), 1 mm acrylic

Still to get 🛒

- Mirror piece 25 × 30 mm (local glass shop)
- Heat-shrink or insulating tape

Stretch (later)

- ESP32-CAM + programmer
- MT3608 boost

Where the files are

What	Path
Test sketches	`firmware/tests/t1…t4`
Send text from Mac	`tools/send_test.py`
Short guide	`docs/hardware_bringup.md`
Daily log	`docs/handoff/YYYY-MM-DD.md`

Step 1: Check the boards with the meter ✅ DONE

Goal: make sure both ESP32-C3 boards give the right voltages.

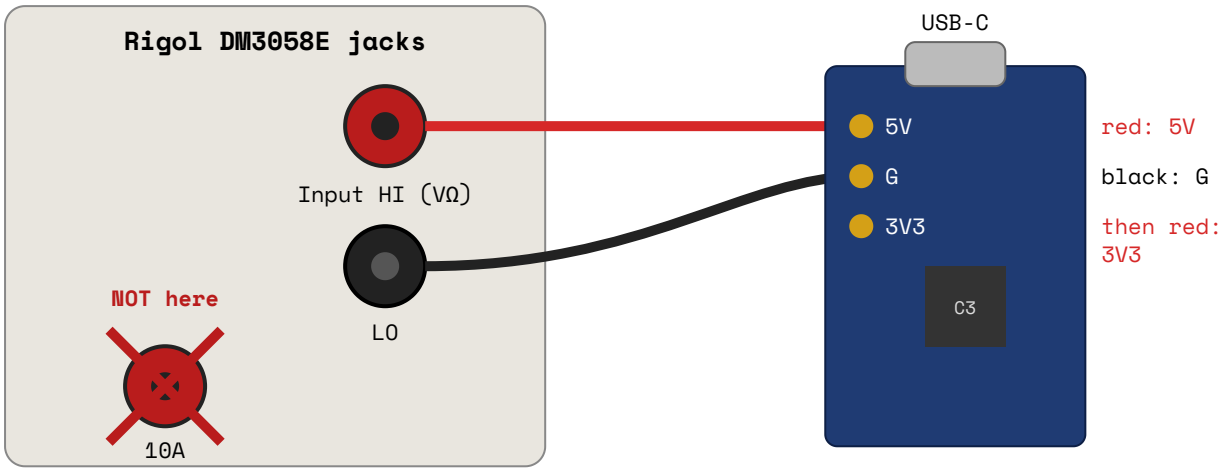


Diagram: red lead in Input HI, black in LO, the 10A jack is never used for volts.

1. Red lead into **Input HI (VΩ)**. Black lead into **LO**.
2. Press the ⎓**V** button (DC volts). It turns green.
3. Plug the ESP32-C3 into the Mac with the Type-C cable.
4. Black probe on the **G** pad.
5. Red probe on the **5V** pad. Read the number.
6. Keep black on G. Move red to **3V3**. Read the number.
7. Repeat for board #2.

Done when
5V reads 4.8–5.2 V and 3V3 reads 3.2–3.4 V on both boards.
Your result: 5V = **5.162 V**, 3V3 = **3.302 V** on both boards ✅

If it fails
Reads about 0 V → the red lead is in the 10A jack; move it to Input HI. No reading → try another USB port or cable.

Step 2: Put the ESP32-C3 on the breadboard (no solder) DONE

Goal: the board sits on pins in the breadboard without soldering. Soldering comes at the very end (Step 13).

2a. Snap the pins DONE



Diagram: snap the strip after every 8th pin to get two pieces.

2b. Pins into the board

1. Hold the ESP32-C3 with the chip side up.
2. From underneath, push the **short end** of piece 1 into one row of holes.
3. Push piece 2 into the other row. The long pins point down.

2c. Board onto the breadboard

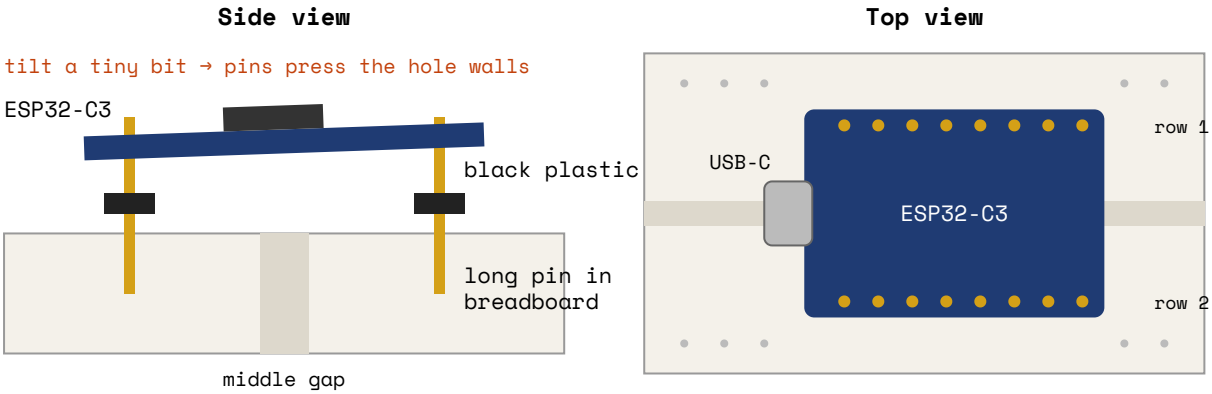


Diagram: one pin row on each side of the middle gap, USB-C toward the breadboard edge.

1. Turn the board so the **USB-C port** points to the edge of the breadboard.
2. Line it up across the **middle gap**: one pin row on each side.
3. Thumbs on the board edges, not on the chip. Press straight down.
4. Tilt it a tiny bit so the pins press against the hole walls.

2d. Check the OLED pins

1. Look at the OLED's 4 pins: **VCC GND SCL SDA** (order may differ, read the labels).
2. Already soldered → plugs straight into the breadboard.
3. Loose pins → same trick as 2b.

Done when
The board sits flat, doesn't wobble, and Step 3 (blink) works with it on the breadboard.

If it fails (later steps)
Random resets, upload errors or "no I2C devices" → **press the board down first**, before blaming the code.

Step 3: Is the ESP32-C3 alive? (blink) ✅ DONE

Goal: upload the first program and see the LED blink. Sketch: `firmware/tests/t1_blink`

3a. One-time setup

1. Install **Arduino IDE 2** from arduino.cc.
2. Open it. Left bar → **Boards Manager**.
3. Search **esp32**. Install **esp32 by Espressif Systems**.

3b. Settings

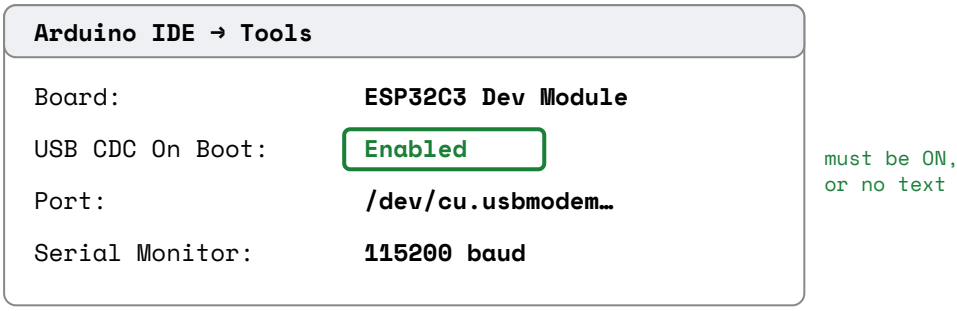


Diagram: the four settings that matter.

1. Plug in the board (USB-C).
2. Set the four settings above.

3c. Upload

1. File → Open → `firmware/tests/t1_blink/t1_blink.ino`.
2. Click **Upload** (the → arrow).
3. Wait for **Done uploading**.
4. Open **Serial Monitor**, set 115200.



Diagram: what you should see.

Done when
The blue LED blinks once per second and Serial Monitor prints `alive 0`, `alive 1`, `…`. Do it for both boards.

If it fails
Upload error → hold **BOOT**, tap **RST**, let go of BOOT, upload again.
No port → try another cable (some only charge).
No text → USB CDC On Boot is not Enabled.
Random disconnects → press the board down (friction fit).

Step 4: Connect the OLED screen ✅ DONE

Goal: the ESP32-C3 finds the OLED, then shows text on it.

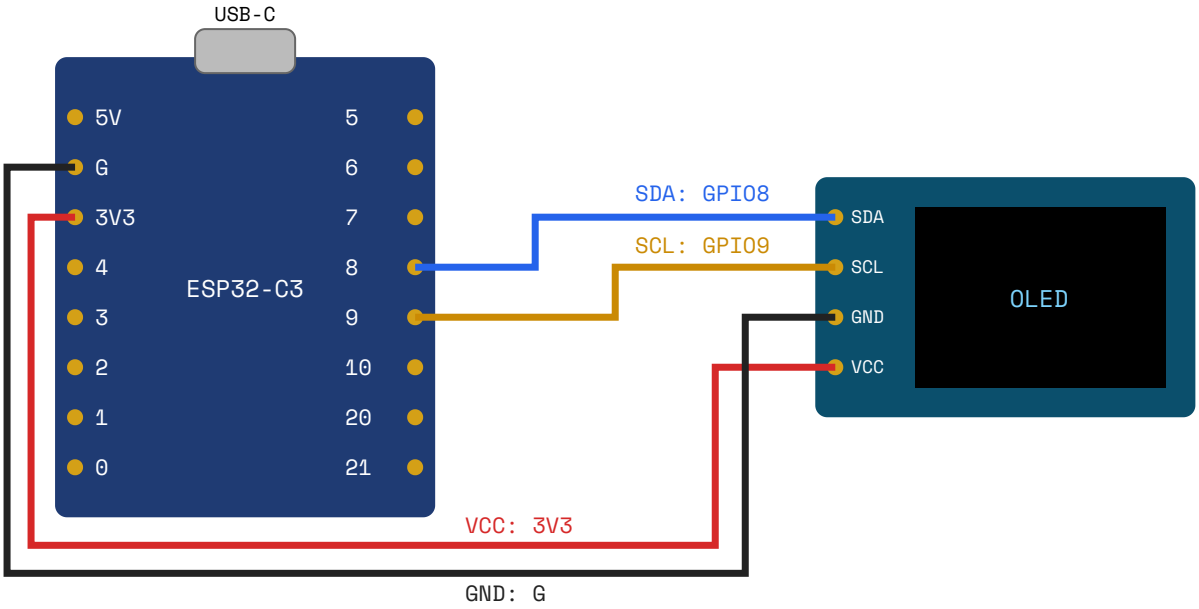


Diagram: red VCC→3V3, black GND→G, blue SDA→GPIO8, yellow SCL→GPIO9. Check the pin labels printed on your own boards.

OLED pin	ESP32-C3 pin	Wire colour
VCC	3V3 (not 5V)	red
GND	G	black
SDA	GPIO8	blue
SCL	GPIO9	yellow

4a. Wire it

1. Unplug USB first.
2. Push the OLED into the breadboard, away from the ESP32-C3.
3. Add the 4 jumper wires from the table. Use the free holes in the same column as each pin.
4. Check every wire twice. Then plug USB back in.

4b. Find the OLED (`t2_i2c_scan`)

1. Upload `firmware/tests/t2_i2c_scan`.
2. Open Serial Monitor.

```
found device at 0x3C
---
```

4c. Show text (`t3_oled_hello`)

1. Library Manager → install **Adafruit SSD1306** (say yes to Adafruit GFX).
2. Upload `firmware/tests/t3_oled_hello`.
3. Mirror test: set `#define MIRROR_X 1`, upload, look at the OLED in a mirror.
4. Swap in OLED #2 and repeat 4b + 4c.



Diagram: normal screen (left), mirrored screen (right).

Done when
Scan shows `0x3C`, the screen shows **HELLO** + a counter, and mirrored text reads correctly in a mirror. Both OLEDs.

If it fails
"no I2C devices" → SDA/SCL swapped, loose wire, or VCC on 5V. Press the board down.
Scan shows `0x3D` → change `OLED_ADDR` to `0x3D` in t3 and t4.
Screen blank but scan OK → Adafruit SSD1306 library missing or wrong address.

Step 5: Connect the ESP32-C3 to Wi-Fi ✅ DONE

Goal: the glasses board joins your phone hotspot and shows its IP address. Sketch: `firmware/tests/t4_wifi_udp`

5a. Phone hotspot

1. Turn on your phone hotspot.
2. iPhone: Settings → Personal Hotspot → **Maximize Compatibility ON** (2.4 GHz).
3. Android: hotspot band = **2.4 GHz**.

5b. Wi-Fi password file

1. Open folder `firmware/tests/t4_wifi_udp/`.
2. Copy `secrets.h.example` → name the copy `secrets.h`.
3. Type your hotspot name and password inside.
4. `secrets.h` is never committed (gitignored). Keep it that way.

```
#define WIFI_SSID "your-hotspot-name"
#define WIFI_PASS "your-hotspot-password"
```

5c. Upload

1. Upload `t4_wifi_udp`.
2. Watch the OLED.

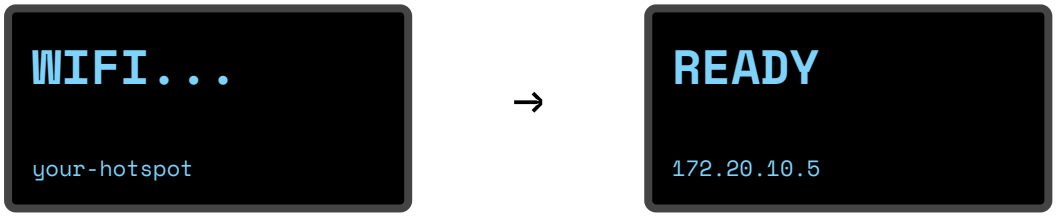


Diagram: connecting (left), connected with its IP address (right). Your IP will differ.

3. Write down the IP shown on the OLED.

Done when
The OLED shows **READY** and an IP address.

If it fails
Stuck on **WIFI...** → wrong password, hotspot is 5 GHz, or the board is too far from the phone. The SuperMini antenna is weak; keep the phone within 1–2 m.

Step 6: Send a word from the Mac to the OLED ✅ DONE

Goal: a word typed on the laptop shows up on the glasses screen. This is the same link the sign recogniser will use.

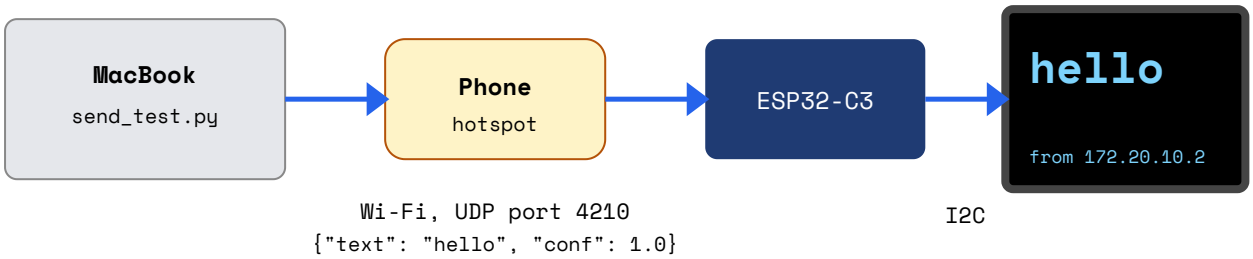


Diagram: the word travels Mac → hotspot → ESP32-C3 → OLED.

1. Connect the Mac to the **same** phone hotspot.
2. Open Terminal in the project folder.
3. Send one word (use the IP from Step 5):

```
venv/bin/python tools/send_test.py --host 172.20.10.5 --text hello
```

4. Look at the OLED.
5. Try the name instead of the IP:

```
venv/bin/python tools/send_test.py --text hello
```

6. Try typing many words (Ctrl+C to stop):

```
venv/bin/python tools/send_test.py --host 172.20.10.5
```

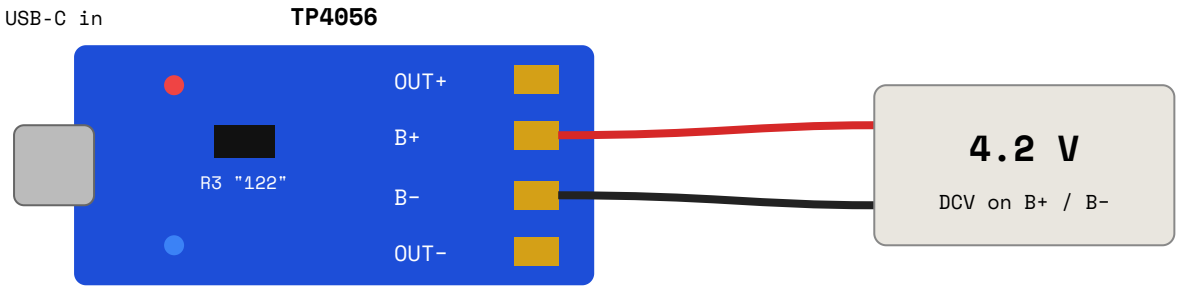
Done when
hello appears on the OLED in under a second.

If it fails
Nothing appears → Mac is on a different Wi-Fi; check the IP again.
IP works but `glasses.local` doesn't → fine for now, use the IP and note it in the daily log.

Step 7: Test the chargers and switches (USB only)

Goal: both TP4056 chargers and all 3 slide switches work. **No battery.**

> ⚠ **LiPo already on the charger? (red LED on)**
> 1. Unplug the USB-C cable. 2. Take the battery wires off the pads and keep red and black apart. 3. Touch the cell: warm or puffy → put it aside on a tile and don't use it. Cool and flat → OK, wait for Step 9.



R3 sets charge current: 1.2 kΩ = 1 A (too high for 600 mAh, we charge with a bench supply instead)

Diagram: measure B+ (red) and B- (black) with only USB plugged in.

7a. Chargers

1. Plug USB-C into TP4056 #1. Nothing else connected.
2. LEDs may flicker or both glow. That's normal with no battery.
3. Meter on DC volts. Red on **B+**, black on **B-**. Read.
4. Red on **OUT+**, black on **OUT-**. Read (0 V is OK without a battery).
5. Repeat for TP4056 #2.

7b. Slide switches

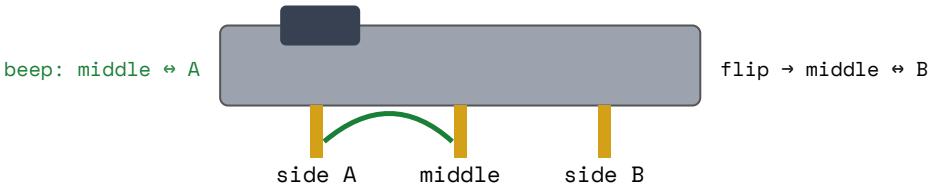


Diagram: the middle pin connects to one side, flip it and it connects to the other.

1. Meter on beep mode ·))).
2. Probes on **middle** pin and **side A** → beep (or not).
3. Flip the switch. Now **middle** and **side B** should beep.
4. Test all 3 switches.

> **Done when**
> B+ reads 4.1–4.25 V on both chargers, and every switch swaps sides when flipped.

> **If it fails**
> B+ reads 0 V → bad USB cable or dead board, try the other cable. B+ above 4.3 V → don't use that board for charging.

Next: Step 9 (solder battery wires). Charging is done with the bench supply (Step 8), never through this board's USB.

Step 8: Charge the LiPo with the bench DC supply

Goal: charge safely at 0.3 A / 4.20 V (CC/CV) without changing the TP4056. Only needed when the cell is below ~3.9 V.

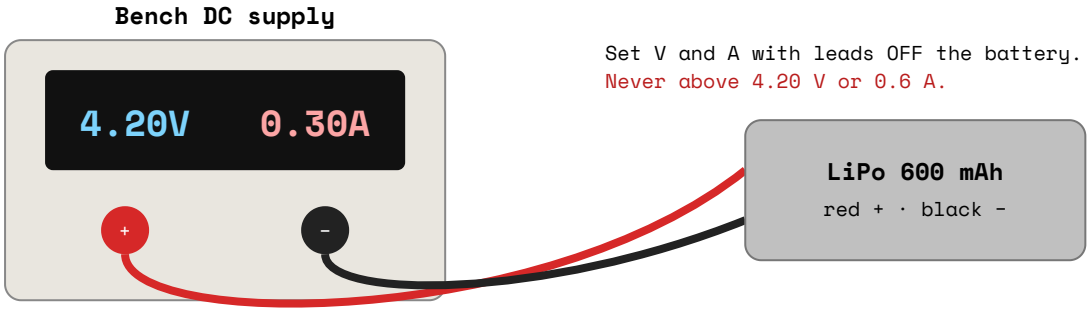


Diagram: supply + → battery red (or the B+ pad), supply − → battery black (or B−).

> ⚠ **Only a lab supply with BOTH a voltage and a current knob**
> A phone adapter, fixed 5 V supply or USB port wired straight to the battery = overcharge and fire. Never.

1. Measure the cell. **3.9 V or more → skip, no charge needed.** Below 3.0 V → use 0.05 A until it passes 3.0 V.
2. Leads **not** on the battery. Supply ON. Set **4.20 V**. Check the leads with your meter: 4.18–4.20 V.
3. Touch the two leads together briefly and set the current limit to **0.30 A**.
4. **OUTPUT OFF.**
5. Clip red → battery red (or B+), black → battery black (or B−). Keep the clips apart. Cell on a tile.
6. **OUTPUT ON.** Display shows ~0.30 A (CC lamp). Voltage climbs slowly.
7. Stay next to it. Touch the cell every 10 minutes. Warm → OUTPUT OFF, stop.
8. Near full the supply switches to CV and the current drops. At **0.03–0.05 A → OUTPUT OFF**, unclip. (~1.5–2.5 h from half full.)
9. Wait 5 min, measure: **4.15–4.20 V** = full.

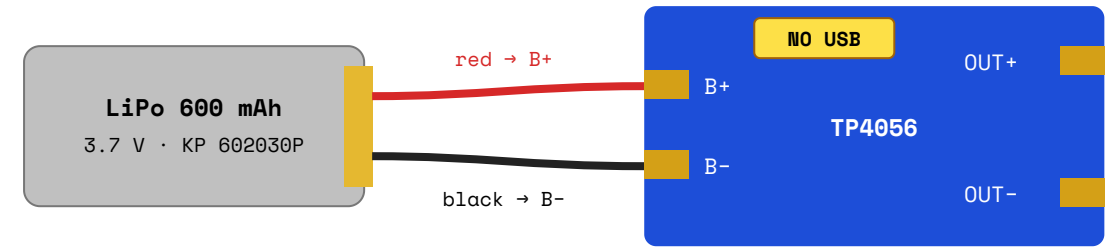
> **Done when**
> Current tapered to ≤ 0.05 A, cell reads 4.15–4.20 V after resting, never got warm.

> **If it fails**
> Current stays at 0 A → clip not touching, or OUTPUT is off. Current stays at 0.30 A for over 3 h → stop, check the voltage setting. Cell warm or puffy → stop and don't use that cell.

Optional later: to charge through the TP4056's USB instead, replace its R3 (`122`, 1.2 kΩ = 1 A) with 2.2–2.4 kΩ (≈ 500 mA). Not needed with the bench supply.

Step 9: Solder the battery wires to the TP4056

Goal: LiPo soldered to B+/B- the right way round, and the TP4056 USB port blocked so it can never charge at 1 A.



Solder the wires. Never hold them on the pads by hand.

Diagram: battery on B+/B- only. The load (ESP32) goes on OUT+/OUT- in Step 10.

9a. Check the cell

1. Meter on DC volts. Red probe on the red wire end, black on the black wire end. Don't let the wire ends touch each other.
2. **3.0–4.2 V** → OK. Below 3.0 V → charge it first with the bench supply (Step 8, 0.05 A start).
3. Cell must be flat, not puffy, not dented.

9b. Solder the wires

1. Charger USB unplugged. Do **one wire at a time** so the two ends can never touch.
2. Tin the B+ pad. Tin the red wire end. Hold the wire on the pad, touch the iron 1–2 s. Same for black → B-.
3. Don't heat the battery itself. Keep the iron on the pad, not the cell.
4. Check again: red goes to **B+**, black goes to **B-**. Wrong way round can kill the board or heat the cell.
5. Put tape over the joints.

9c. Block the USB port

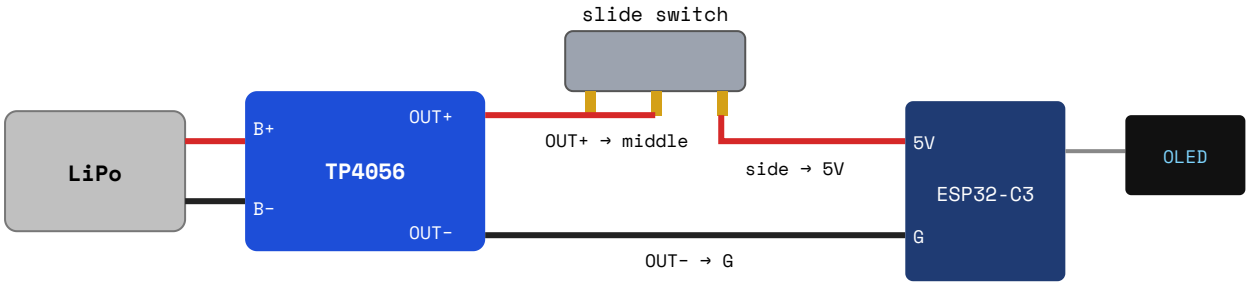
1. Put tape over the TP4056 USB-C port. Write **NO USB** on it.
2. Measure OUT+/OUT- → ≈ battery voltage. (0 V → the protection chip is asleep; see Step 10 "If it fails".)
3. To charge later: leave the cell soldered, clip the bench supply to **B+/B-** (Step 8).
4. Repeat 9a–9c for LiPo #2 on TP4056 #2.

Done when
Both cells soldered red → B+, black → B-, joints taped, USB ports taped "NO USB".

If it fails
Wire came off → re-tin pad and wire. Board got hot while soldering a cell → unsolder, check the cell isn't puffy.

Step 10: Run the ESP32-C3 from the battery

Goal: the glasses electronics work with no USB cable: battery → charger → switch → ESP32-C3 → OLED.



Never plug the ESP32's USB-C while the switch is ON (two supplies on the 5V pin).

Diagram: OUT+ → switch middle, switch side → ESP32-C3 5V. OUT- → G. OLED wiring same as Step 4.

1. Switch OFF. ESP32 USB-C unplugged. OLED still wired as in Step 4. TP4056 USB stays taped.
2. Jumper: **OUT+** → switch **middle**. Switch **side** → ESP32-C3 **5V**.
3. Jumper: **OUT-** → ESP32-C3 **G**.
4. Phone hotspot on. Switch ON. OLED shows `WIFI...` then `READY` + IP (`t4_wifi_udp` is still flashed).
5. Meter on DC volts: ESP32 3V3 to G → ≈ 3.3 V.
6. Mac on the hotspot: `venv/bin/python tools/send_test.py --host <IP> --text battery` → word shows on the OLED.
7. Flip the switch OFF → everything goes dark. ON → it boots again.
8. **Runtime test:** full cell, switch ON, write the start time. Note when the OLED goes off. That's your battery life. Recharge with the bench supply (Step 8).

Done when
OLED shows READY + IP from the battery, `send_test.py` text appears, switch turns it on/off, runtime written down.

If it fails
Nothing lights → measure OUT+/OUT- (should be ≈ battery voltage). 0 V → touch the bench supply (4.20 V, 0.3 A) to B+/B- for 2 s to "wake" the protection chip. Resets when Wi-Fi connects → cell is low, charge it. Works on USB but not battery → check the switch side you used (Step 7 beep test).

Step 11: Leftover checks from Steps 3–6

Goal: close the small tests that were skipped. USB power is fine here (battery switch OFF).

Test	How	Pass
ESP32-C3 #2 blink	Swap in board #2, upload `t1_blink`	Blue LED blinks, serial shows `alive 0`, `1`, …
OLED #2	Swap in OLED #2, upload `t2_i2c_scan`, then `t3_oled_hello`	`0x3C` found, HELLO + counter
OLED #1 mirror	`t3_oled_hello` with `MIRROR_X 1`, look at it in a mirror	Text reads normally in the mirror
glasses.local	Upload `t4_wifi_udp`, then `venv/bin/python tools/send_test.py --text hello`	hello on the OLED (if not: use the IP, note it)

Done when
All 4 rows pass (glasses.local failing is OK, just write it down).

Step 12: Optics bench test (lens + acrylic)

Goal: see sharp, readable OLED text through the lens and acrylic before building the housing.

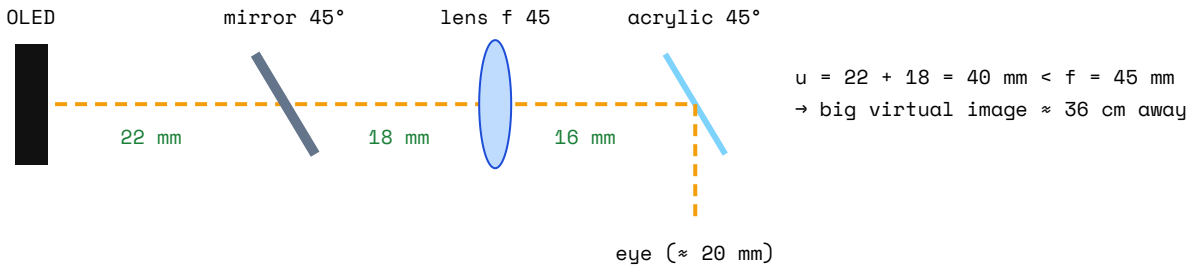


Diagram: light path unfolded. OLED → mirror → lens → acrylic → eye.

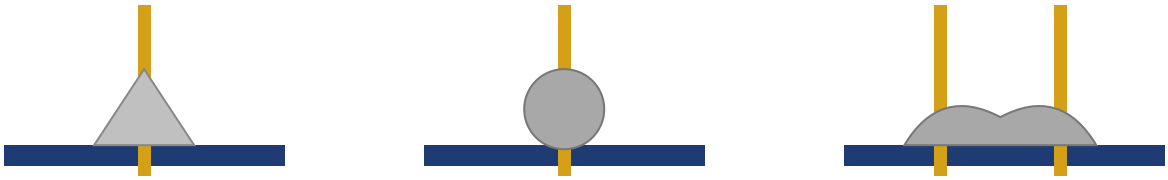
1. Run `t3_oled_hello` on USB (big text is easy to judge).
2. **Without the mirror (you can do this now):** hold the lens 40 mm in front of the OLED (use a ruler). Look through the lens → large, sharp HELLO.
3. Move the OLED ±3 mm until the text is sharpest. Write that distance down.
4. Add the acrylic at 45° behind the lens. Look into it from the side → you see the text floating in front of you.
5. Text backwards? Set `MIRROR_X` to the other value, upload, look again.
6. **When the mirror is cut (25 × 30 mm):** fold the path with it (22 mm OLED → mirror, 18 mm mirror → lens) and repeat 3–5.
7. Hard to see in a bright room → raise the contrast, put dark paper behind the acrylic.

Done when
Text is sharp and reads the right way round. Best u (mm) and the `MIRROR_X` value written down.

If it fails
Double image → acrylic too thick, use the 1 mm one. Blurry at every position → u is too close to 45 mm; move the OLED closer to the lens.

Step 13: Final build: soldering LATER

Goal: permanent, clean pins. Do this only after Steps 7–12 pass.



✓ GOOD
shiny cone

✗ BALL
reheat 1 s

✗ BRIDGE
two pins joined

Diagram: a good joint, a ball (not enough heat), and a bridge (short circuit).

1. Pins in the breadboard, board on top (same as Step 2). This keeps them straight.
2. Heat the iron for 2 minutes (~330 °C). Wipe the tip on the wet sponge.
3. Touch the iron to the **pin and the copper ring together**. Count 1 second.
4. Push solder onto the joint (not the iron) until a small cone forms.
5. Remove the solder first, then the iron.
6. Do one corner pin, check the board is flat, then the rest.
7. Unplug the iron. Let the board cool for 1 minute.
8. Beep mode: test every pair of neighbouring pins → **no beep**.
9. Run Steps 3 and 4 again.
0. Final wires (short, thin): LiPo → TP4056 → switch → ESP32 5V/G, OLED → 3V3/G/GPIO8/GPIO9. Run Step 10 again.

Done when
All joints are shiny cones, no neighbouring pins beep, and blink + OLED + battery power work again.

If it fails
Bridge → add flux, drag the clean hot tip between the pins. Ball → reheat with a touch of flux.

Step 14: Enclosure + mount on the glasses LATER

Goal: everything inside the 3D-printed housing, clipped to the right temple arm.

1. Measure: temple arm width × height where the clip goes; LiPo, TP4056, ESP32-C3, OLED, mirror thickness.
2. Print `hardware/enclosure/stl/test_fit.stl` first (college lab). Check the parts fit; tune `clr` in `hud_enclosure.scad`.
3. Enclosure v2 (pockets + clip-on mount): plan in `docs/handoff/2026-10-29.md`. Build it, then print `body`, `lid`, `lens_plate`, `clip`.
4. Paint the inside matte black. Fit mirror, lens, acrylic. Keep the optics distances from Step 12.
5. Assembly order and print settings: `hardware/enclosure/README.md`.
6. Charger USB-C and switch must be reachable from outside.

Done when
Glasses on, switch ON, `send_test.py` word appears sharp in front of your eye.

Step 15: Results checklist

Tick each box as you go. Copy the results into `docs/hardware_bringup.md` and today's daily log.

#	Part	Test (step)	OK	Notes / reading
1	ESP32-C3 #1	Volts 5V / 3V3 (1)	☑	5.162 V / 3.302 V
2	ESP32-C3 #1	Friction fit + blink + serial (2, 3)	☑	2026-10-25
3	ESP32-C3 #2	Volts 5V / 3V3 (1)	☑	5.162 V / 3.302 V
4	Type-C cable	Upload works (3)	☑	
5	Breadboard + jumpers	No faults (4–6)	☑	after wiring fix
6	OLED #1	Scan 0x3C, HELLO (4)	☑	text + bitmap
7	Wi-Fi + UDP	send_test.py → OLED via IP (5, 6)	☑	10.249.236.231
8	TP4056 #1	B+ ≈ 4.2 V, USB only (7)	☐	V:
9	TP4056 #2	B+ ≈ 4.2 V, USB only (7)	☐	V:
10	Slide switches ×3	Middle ↔ side swaps (7)	☐	
11	Bench supply	Set 4.20 V / 0.30 A, checked with meter (8)	☐	V:
12	LiPo charge	Bench charge, tapered, not warm (8, when needed)	☐	end V:
13	LiPo #1	Volts as received (9a)	☑	3.999 V (2026-10-29)
14	LiPo #2	Volts as received (9a)	☐	V:
15	LiPo #1 + TP4056 #1	Soldered, taped, NO-USB tape (9)	☐	OUT V:
16	LiPo #2 + TP4056 #2	Soldered, taped, NO-USB tape (9)	☐	OUT V:
17	Battery power	READY + send_test.py, switch on/off (10)	☐	3V3:
18	Battery life	Runtime test (10)	☐	hours:
19	ESP32-C3 #2	Blink + serial (11)	☐	
20	OLED #2	Scan 0x3C, HELLO (11)	☐	
21	OLED #1	Mirror check (11)	☐	
22	glasses.local	send_test.py without IP (11)	☐	yes / no
23	Optics	Sharp, right way round (12)	☐	u = mm, MIRROR_X =
24	Soldering	Final build (13)	☐	
25	Enclosure	test_fit, print, assemble (14)	☐	

What comes after

1. Full glasses firmware `firmware/glasses_display/glasses_display.ino`: word-wrap, history, "NO LINK" screen.
2. Connect the sign-recognition app to the same UDP link.